Classes e Objetos

Projeto: Sistema de Projetos e Portfólio
Pré-requisito: Aula 1 — Java, ambiente e arquitetura do projeto

Nesta aula, o aluno começa a deixar de trabalhar apenas com comandos e passa a construir **objetos que representam elementos reais do sistema**.

A ideia central é:

> **Na Aula 1, criamos um programa Java. Na Aula 2, começamos a construir o sistema.**

---

1. Objetivos da aula

Ao final da aula, o aluno deverá ser capaz de:

- compreender o conceito de **classe**;
- compreender o conceito de **objeto**;
- identificar atributos e comportamentos;
- criar uma classe em Java;
- criar objetos a partir de uma classe;
- utilizar construtores;
- utilizar métodos;
- compreender `private`;
- utilizar `getters` e `setters`;
- compreender encapsulamento;
- criar a primeira classe real do projeto: `Projeto`.

2. Onde estamos no projeto?

Na Aula 1 tínhamos:

```
SistemaProjetos/
│
└── src/
    └── Main.java
```

Agora começaremos a construir:

```
SistemaProjetos/
│
└── src/
    │
    ├── Main.java
    │
    └── model/
        └── Projeto.java
```

A classe `Projeto` será uma das classes fundamentais do sistema.

Posteriormente teremos:

```
Projeto
   │
   ▼
ProjetoService
   │
   ▼
ProjetoCSV
   │
   ▼
API REST
   │
   ├──────────► Web
   │
   └──────────► Desktop
```

Portanto, **a classe criada nesta aula será utilizada durante praticamente todo o restante do curso**.

3. Início da aula: pensando como programador

Antes de apresentar código, faça uma pergunta:

> **Se quisermos criar um sistema para cadastrar projetos, o que precisamos armazenar sobre um projeto?**

Provavelmente aparecerão:

- nome;
- descrição;
- categoria;
- responsável;
- data;
- status;
- endereço;
- código identificador.

No nosso primeiro modelo, vamos trabalhar com:

```
Projeto

id
nome
descricao
categoria
status
```

> "Estamos tentando transformar uma coisa do mundo real em uma estrutura que o computador consiga representar."

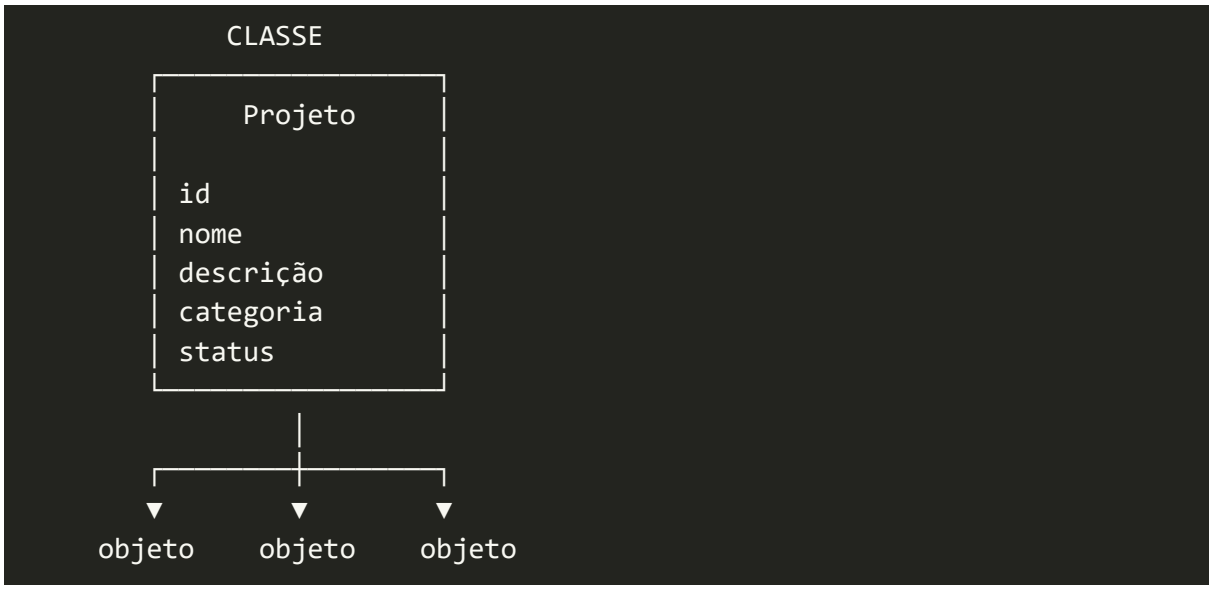
Esse é o princípio da **modelagem orientada a objetos**.

4. O que é uma classe?

Uma forma simples de explicar:

> **Classe é um modelo ou molde para criar objetos.**

Analogia:



A classe define **como será um objeto**.

---

5. Classe não é objeto

Essa diferença deve ficar muito clara.

Classe

É o modelo:

```
Projeto
```

Objetos

São instâncias concretas:

```
Projeto 1
Projeto 2
Projeto 3
```

Por exemplo:

```
Projeto 1
id = 1
nome = "Sistema Acadêmico"
categoria = "Software"
status = "Em desenvolvimento"
```

Outro:

```
Projeto 2
id = 2
nome = "Site Institucional"
categoria = "Web"
status = "Concluído"
```

Os dois são objetos da mesma classe:

```
Projeto
```

6. Criando a primeira classe

Crie a pasta:

```
src/model
```

E dentro dela:

```
Projeto.java
```

Começaremos de maneira extremamente simples:

```
package model;

public class Projeto {

}
```

Explique:

```
public class Projeto
```

significa:

> "Estamos criando uma classe chamada Projeto."

E:

```
package model;
```

indica que a classe pertence ao pacote `model`.

---

7. Atributos

Agora precisamos definir as características de um projeto.

```
package model;

public class Projeto {

    private int id;
    private String nome;
    private String descricao;
    private String categoria;
    private String status;

}
```

Temos agora cinco atributos:

```
Projeto
│
├── id
├── nome
├── descricao
├── categoria
└── status
```

Explique:

`int`

Utilizado para números inteiros.

```
private int id;
```

`String`

Utilizado para textos.

```
private String nome;
```

8. Por que utilizar `private`?

Este é o primeiro contato com **encapsulamento**.

> "Queremos proteger os dados internos do objeto e controlar a maneira como eles são acessados."

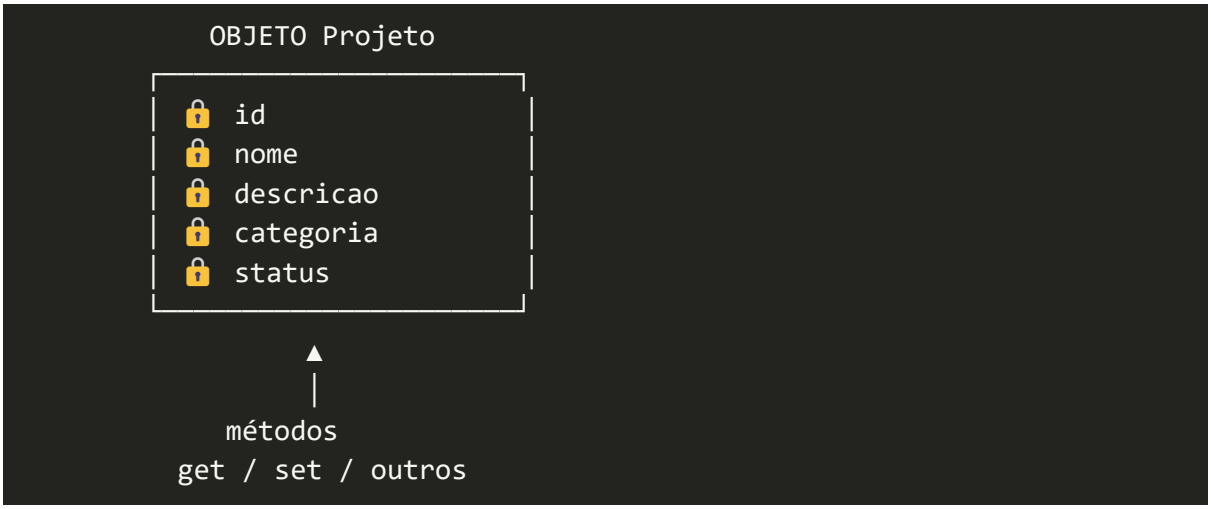
Por isso:

```
private int id;
private String nome;
```

em vez de:

```
public int id;
public String nome;
```

Visualmente:



A ideia é preparar o aluno para entender que **o objeto controla seus próprios dados**.

---

9. Criando o primeiro objeto

Agora vamos voltar ao `Main.java`.

```java
import model.Projeto;

public class Main {

    public static void main(String[] args) {

        Projeto projeto = new Projeto();

    }
}
```

Explique cuidadosamente:

```java
Projeto projeto
```

Estamos declarando uma variável capaz de armazenar um objeto `Projeto`.

E:

```java
new Projeto()
```

estamos criando efetivamente o objeto.

Uma boa representação:

```
Projeto projeto = new Projeto();
          |                |
          |                └── cria objeto
          |
          └── variável que referencia o objeto
```

10. O conceito de instância

Explique:

> Um objeto criado a partir de uma classe é uma **instância** dessa classe.

Podemos criar vários:

```
Projeto projeto1 = new Projeto();
Projeto projeto2 = new Projeto();
Projeto projeto3 = new Projeto();
```

Temos:



Cada objeto possui seus próprios dados.

---

11. Getters e Setters

Agora precisamos conseguir acessar os atributos.

Crie:

```
package model;

public class Projeto {

    private int id;
    private String nome;
    private String descricao;
    private String categoria;
    private String status;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
```

```
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getDescricao() {
        return descricao;
    }

    public void setDescricao(String descricao) {
        this.descricao = descricao;
    }

    public String getCategoria() {
        return categoria;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

12. Como funciona o `set`

No `Main`:

```
Projeto projeto = new Projeto();

projeto.setId(1);
projeto.setNome("Sistema Acadêmico");
projeto.setDescricao("Sistema para gerenciamento acadêmico");
projeto.setCategoria("Software");
projeto.setStatus("Em desenvolvimento");
```

Agora o objeto possui informações.

Podemos imaginar:

```
projeto
│
├── id = 1
├── nome = "Sistema Acadêmico"
├── descricao = "Sistema para gerenciamento acadêmico"
├── categoria = "Software"
└── status = "Em desenvolvimento"
```

13. Como funciona o `get`

Agora podemos recuperar os valores:

```
System.out.println(projeto.getId());
System.out.println(projeto.getNome());
System.out.println(projeto.getDescricao());
System.out.println(projeto.getCategoria());
System.out.println(projeto.getStatus());
```

Resultado:

```
1
Sistema Acadêmico
Sistema para gerenciamento acadêmico
Software
Em desenvolvimento
```

14. Entendendo `this`

Este é um ponto que costuma gerar dúvidas.

```
public void setNome(String nome) {
    this.nome = nome;
}
```

Temos dois `nome`:

```
this.nome
   │
   └── atributo do objeto

nome
   │
   └── parâmetro recebido pelo método
```

Portanto:

```
this.nome = nome;
```

significa:

> "Coloque o valor recebido no atributo `nome` deste objeto."

Faça uma pequena demonstração:

```
Projeto projeto1 = new Projeto();
Projeto projeto2 = new Projeto();

projeto1.setNome("Sistema Acadêmico");
projeto2.setNome("Site Institucional");
```

Agora:

```
projeto1
nome = Sistema Acadêmico

projeto2
nome = Site Institucional
```

Cada objeto possui seu próprio estado.

15. Criando um construtor

Até agora fazemos:

```
Projeto projeto = new Projeto();

projeto.setId(1);
projeto.setNome("Sistema Acadêmico");
projeto.setDescricao("Sistema acadêmico");
projeto.setCategoria("Software");
projeto.setStatus("Em desenvolvimento");
```

Podemos tornar isso mais organizado usando um construtor.

Na classe:

```
public Projeto(int id, String nome, String descricao,
               String categoria, String status) {

    this.id = id;
    this.nome = nome;
    this.descricao = descricao;
    this.categoria = categoria;
    this.status = status;
}
```

Agora podemos criar:

```
Projeto projeto = new Projeto(
    1,
    "Sistema Acadêmico",
    "Sistema para gerenciamento acadêmico",
    "Software",
    "Em desenvolvimento"
);
```

Isso é muito mais próximo de como trabalharemos nas próximas aulas.

16. Código completo da classe `Projeto`

Ao final da explicação, o aluno deverá chegar a:

```
package model;

public class Projeto {

    private int id;
    private String nome;
    private String descricao;
    private String categoria;
    private String status;

    public Projeto() {
    }

    public Projeto(int id, String nome, String descricao,
                   String categoria, String status) {

        this.id = id;
        this.nome = nome;
        this.descricao = descricao;
        this.categoria = categoria;
        this.status = status;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getDescricao() {
        return descricao;
    }
```

```
    public void setDescricao(String descricao) {
        this.descricao = descricao;
    }

    public String getCategoria() {
        return categoria;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

17. Testando a classe

O `Main.java` pode ficar assim:

```
import model.Projeto;

public class Main {

    public static void main(String[] args) {

        Projeto projeto1 = new Projeto(
            1,
            "Sistema Acadêmico",
            "Sistema para gerenciamento acadêmico",
            "Software",
            "Em desenvolvimento"
        );

        Projeto projeto2 = new Projeto(
            2,
            "Site Institucional",
```

```
            "Website de uma instituição",
            "Web",
            "Concluído"
        );

        System.out.println("PROJETO 1");
        System.out.println(projeto1.getNome());
        System.out.println(projeto1.getCategoria());
        System.out.println(projeto1.getStatus());

        System.out.println();

        System.out.println("PROJETO 2");
        System.out.println(projeto2.getNome());
        System.out.println(projeto2.getCategoria());
        System.out.println(projeto2.getStatus());
    }
}
```

Resultado esperado:

```
PROJETO 1
Sistema Acadêmico
Software
Em desenvolvimento

PROJETO 2
Site Institucional
Web
Concluído
```

18. Uma pequena evolução: método `exibirDados()`

Agora introduza a ideia de que um objeto pode ter **comportamentos**, não apenas dados.

Na classe `Projeto`:

```
public void exibirDados() {

    System.out.println("ID: " + id);
    System.out.println("Nome: " + nome);
    System.out.println("Descrição: " + descricao);
    System.out.println("Categoria: " + categoria);
    System.out.println("Status: " + status);
}
```

Agora:

```
projeto1.exibirDados();
```

O objeto possui:

Características

```
id
nome
descricao
categoria
status
```

Comportamento

```
exibirDados()
```

Essa distinção é fundamental:

> **Objeto = dados + comportamentos.**

19. Exercício 1 — Criando projetos

Peça ao aluno para criar três objetos:

```
Projeto projeto1;
Projeto projeto2;
Projeto projeto3;
```

Com os seguintes dados:

Projeto 1

```
ID: 1
Nome: Sistema Acadêmico
Categoria: Software
Status: Em desenvolvimento
```

Projeto 2

```
ID: 2
Nome: Website Institucional
Categoria: Web
Status: Concluído
```

Projeto 3

```
ID: 3
Nome: Aplicativo Mobile
Categoria: Mobile
Status: Planejado
```

Depois:

```
projeto1.exibirDados();
projeto2.exibirDados();
projeto3.exibirDados();
```

20. Exercício 2 — Alterando um objeto

Depois de criar:

```
Projeto projeto = new Projeto(
    1,
    "Sistema Acadêmico",
    "Sistema acadêmico",
    "Software",
    "Em desenvolvimento"
);
```

altere o status:

```
projeto.setStatus("Concluído");
```

Depois:

```
System.out.println(projeto.getStatus());
```

Resultado:

```
Concluído
```

Pergunte aos alunos:

> **O que mudou?**

Resposta esperada:

> O estado do objeto `projeto` foi alterado.

---

21. Exercício 3 — Desafio

Agora peça que criem uma classe diferente:

```
Servico
```

Com:

```
id
nome
descricao
preco
```

Por exemplo:

```
Servico servico = new Servico(
    1,
    "Desenvolvimento Web",
    "Criação de websites",
    2500.00
);
```

Esse exercício prepara o aluno para perceber que o mesmo raciocínio pode ser utilizado para representar diferentes entidades.

---

22. Exercício 4 — Identificando classes

Apresente o seguinte cenário:

> "Uma empresa deseja um sistema para gerenciar projetos, serviços e contatos."

Pergunte:

Quais poderiam ser as classes?

Os alunos devem chegar a algo próximo de:

```
Projeto
Servico
Contato
Usuario
```

Depois mostre a evolução futura:

```
model/
│
├── Projeto.java
├── Servico.java
├── Contato.java
└── Usuario.java
```

Explique que **não vamos criar todas hoje**.

Primeiro vamos dominar `Projeto`.

---

23. Atividade principal — Modelando o Sistema

Situação-problema

> Uma instituição deseja apresentar seus projetos em um site. Para cada projeto é necessário armazenar código, nome, descrição, categoria e situação.

Tarefa

O aluno deverá:

1. criar a classe `Projeto`;
2. criar os cinco atributos;
3. utilizar encapsulamento;
4. criar construtor;
5. criar getters e setters;
6. criar três objetos;
7. preencher os dados;
8. exibir os dados;
9. alterar o status de um projeto.

---

24. Desafio adicional

Para os alunos que terminarem mais rapidamente:

Crie o método:

```
public boolean estaConcluido() {

    return status.equals("Concluído");
}
```

Depois:

```
if (projeto.estaConcluido()) {
    System.out.println("Projeto concluído!");
} else {
    System.out.println("Projeto ainda não foi concluído.");
}
```

Aqui começamos a introduzir uma ideia importante:

> **As regras relacionadas ao projeto podem ficar dentro da própria estrutura de domínio.**

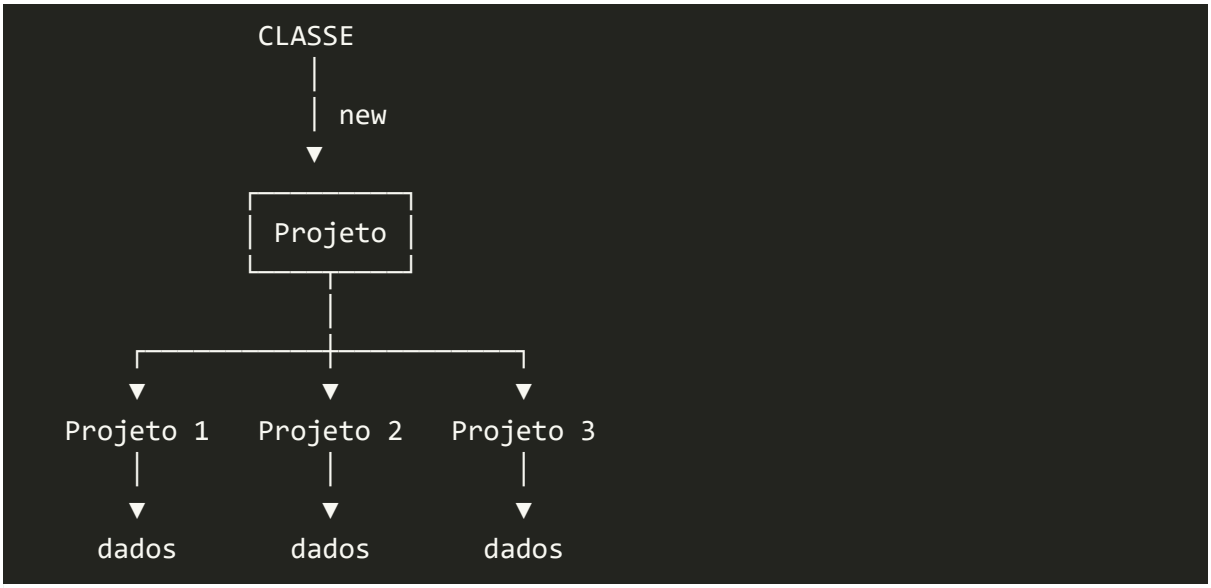
Na Aula 3 isso evoluirá para uma camada de **regras de negócio/Service**.

25. Estrutura final da Aula 2

Ao terminar a aula:

```
SistemaProjetos/
│
├── src/
│   │
│   ├── Main.java
│   │
│   └── model/
│       └── Projeto.java
│
└── README.md
```

E conceitualmente:



26. Fechamento da aula

Faça estas perguntas oralmente:

1. O que é uma classe?

Resposta esperada: um modelo utilizado para criar objetos.

2. O que é um objeto?

Resposta: uma instância de uma classe.

3. O que são atributos?

Resposta: características/dados de um objeto.

4. O que são métodos?

Resposta: comportamentos/operações que podem ser executados.

5. Para que serve `private`?

Resposta: restringir o acesso direto aos atributos e favorecer o encapsulamento.

6. Para que servem `get` e `set`?

Resposta: acessar e modificar os atributos de forma controlada.

7. Para que serve `new`?

Resposta: criar uma nova instância de uma classe.

---

27. Conexão com a próxima aula

Finalize mostrando:

```
AULA 1
Java funciona
      |
      ▼
AULA 2
Temos objetos
      |
      ▼
AULA 3
Temos vários objetos + regras
      |
      ▼
AULA 4
Salvamos objetos no CSV
      |
      ▼
AULA 5
Construímos o CRUD
```

A frase de fechamento pode ser:

> **"Hoje criamos o primeiro elemento real do nosso sistema: o objeto Projeto. Na próxima aula, vamos aprender a trabalhar com muitos projetos ao mesmo tempo e começar a criar as regras de negócio."**